

AER1217: Development of Unmanned Aerial Vehicles

Final Project: Autonomous Drone Fire Fighting

1 Introduction

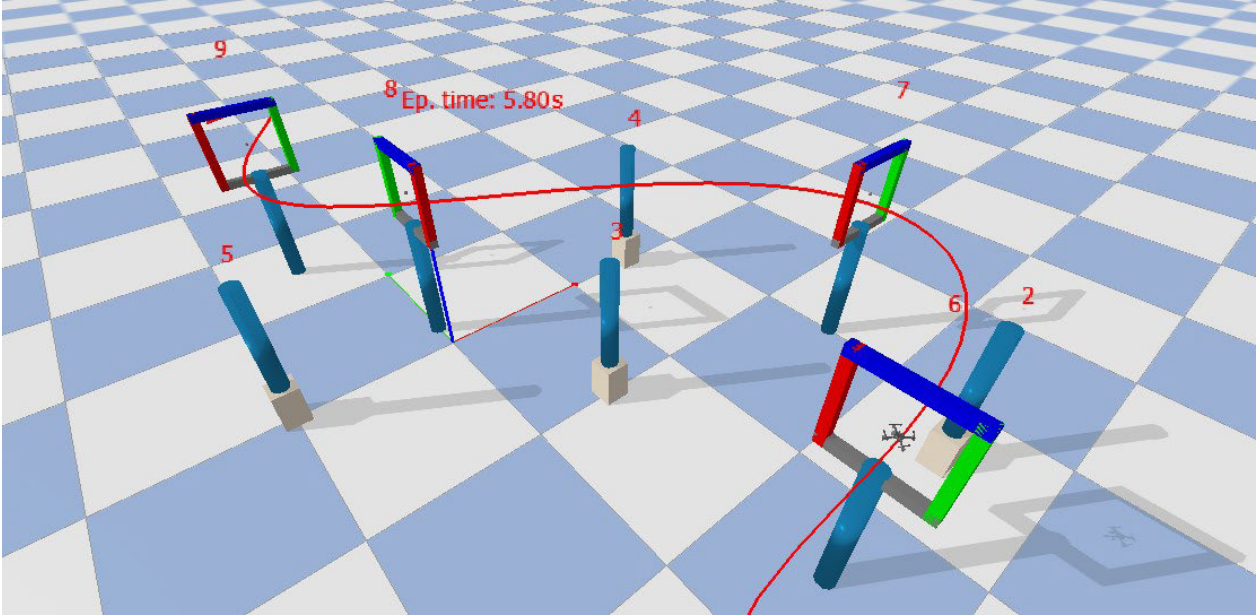


Figure 1: Final project PyBullet simulation environment for autonomous drone firefighting. A Crazyflie nano-quadrotor is required to fly through the gates (fire spots) and reach the target point as accurately as possible, without collisions.

Autonomous firefighting has become an increasingly important application in which unmanned aerial vehicles are deployed to navigate complex and hazardous environments at high speeds. By integrating cutting-edge robotics technology with real-time perception and decision-making, aerial fire-fighting systems are opening new possibilities for rapid response in physical environments, simulated scenarios, and digital twins.

Human fire-fighting operations require extensive training and rely heavily on experience to handle dynamic and uncertain conditions. Today, robotics researchers are developing autonomous algorithms that aim to assist or outperform human-operated systems in tasks such as fire detection, navigation, and suppression. A key challenge is planning a time-optimal and safe trajectory that allows the vehicle to reach critical locations while satisfying environmental, safety, and dynamic constraints. In this final project, we focus on an autonomous firefighting scenario, emphasizing the motion planning and trajectory optimization problems in robotics.

The final project includes two phases:

- **Phase 1:** In the first phase, you are given a [PyBullet](#) simulation environment to develop the algorithm. The task is to design a planning algorithm to navigate, as accurately as possible, through the gates (active



fires) and reach the target point while avoiding obstacles (mountains and buildings). The sequence of the gates will be provided during the project demonstration day (gates could be repeated). In the simulation, we provide accurate pose measurements of the quadrotor, positions of the gates, interfaces to a low-level controller, and noisy positions of the obstacles. You are required to show that the proposed algorithm can safely navigate the quadrotor through the gates while dealing with the uncertainties of the obstacles.

- **Phase 2:** In the second phase, we will execute the algorithm you developed in a real-world experimental setup. We will check if the proposed algorithm can navigate safely through the gates with a predefined sequence accurately and quickly.

2 Simulation Environment Setup

The PyBullet simulation environment we provided is based on the open-source project [safe-control-gym](#). We recommend Ubuntu 22.04 on a mid-tier laptop. Open Terminal, clone the git repository, and check out to the `aer1217-course-project` branch:

```
mkdir aer1217-project
cd aer1217-project
git clone https://github.com/FSC-Lab/AER1217_2026.git
cd safe-control-gym
git checkout aer1217-course-project
```

While not mandatory, we recommend using [Anaconda](#) to manage the software environment. Create and access a Python 3.8 environment using conda:

```
conda create -n aer1217-project python=3.8
conda activate aer1217-project
```

Install the `safe-control-gym` repository:

```
pip install --upgrade pip
pip install -e .
```

Now, install `pycffi-firmware`, which is the Python bindings of the official Crazyflie firmware. Note, `pycffi-firmware` needs to be installed in the same folder as `safe-control-gym`:

```
cd ..
git clone https://github.com/utiasDSL/pycffi-firmware.git
cd pycffi-firmware/
git submodule update --init --recursive
```

```
sudo apt update
sudo apt -y install swig
sudo apt install build-essential
cd wrapper/
chmod +x build_linux.sh
./build_linux.sh
```

Then, run the scripts in the aer-course-project/ folder:

```
cd ../../safe-control-gym/aer-course-project/
python3 final_project.py --overrides ./getting_started.yaml
```

3 Project Description

3.1 Gates and Obstacles

An overhead view of the simulation environment and its schematic diagram is provided in Figure 2. There are four gates (fire spots to be extinguished) and four obstacles (buildings or mountains that drones cannot fly across) in the environment. The quadrotor is required to navigate through all the gates before reaching the target point.

The accurate positions of the gates and the nominal positions of the obstacles are provided in Table 1. The gates and obstacles variables should be modified accordingly in gettingstarted.yaml. We injected uniform noises $U(-0.2, 0.2)$ to the obstacles' nominal x and y positions to represent the uncertainties of the environment.

The starting and final target points are $(-1.0, -3.0, 0.0)$ and $(-0.5, 2.0, 0.0)$, respectively.

Table 1: Coordinates of the center of the Four Gates

Gate ID	x (m)	y (m)	z (m)	Direction
Gate 1	0.5	-2.5	1.0	Vertical
Gate 2	2.0	-1.5	1.2	Horizontal
Gate 3	0.0	0.5	0.8	Vertical
Gate 4	-0.5	1.5	1.9	Horizontal

The dimensions of the gates and obstacles are shown in Figure 3. The inner edges of the square gates are 40 cm in length. The obstacles are pillars with a 12 cm diameter and 1.3 m in height.

Passing through the gates from either direction without collision counts as one successful pass. Repeatedly passing the same gate only counts for one pass.

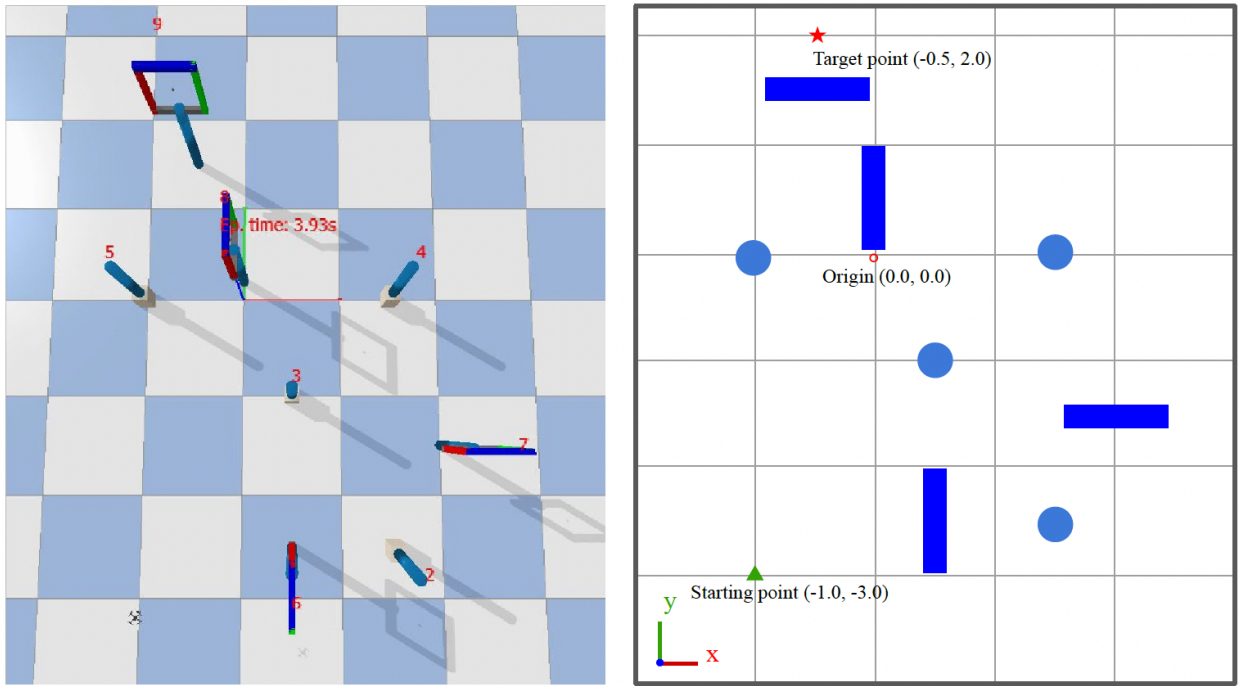


Figure 2: (Left) Overhead view of the simulation environment. (Right) Schematic diagram of the simulation environment. Obstacles and gates are indicated as circles and squares, respectively.

3.2 Trajectory Planning

As described above, the x and y positions of the obstacles are corrupted with uniform noise. Hence, the designed planning algorithm should take into account the uncertainties to ensure safe navigation.

The sequence of the gates will be provided on the project demonstration day. You are required to plan the trajectory based on the provided sequence and navigate the quadrotor in simulation and physical experiments.

We will record the total tracking error the drone experienced and time it takes for the quadrotor to finish the trajectory and reach the final target point in experiments. As a drone racing competition, we will rank the teams based on their flight time in completing the task, which will account for 5% of the final score.

The environment boundaries are set to:

$$x \in [-3.5, 3.5], \quad y \in [-3.5, 3.5], \quad z \in [0, 2].$$

The planned trajectory should be within this range. Please do not fly outside these boundaries.

3.3 Controller Interface

In the simulation environment, we provide the [Crazyswarm's](#) Crazyflie command interface, including high-level commands `takeoff()`, `land()`, `goTo()`, and low-level full-state commands `cmdFullState()`.

The low-level command `cmdFullState()` sends the desired position, velocity, acceleration, and heading references to the onboard controller, which is beneficial for smooth trajectory planning.

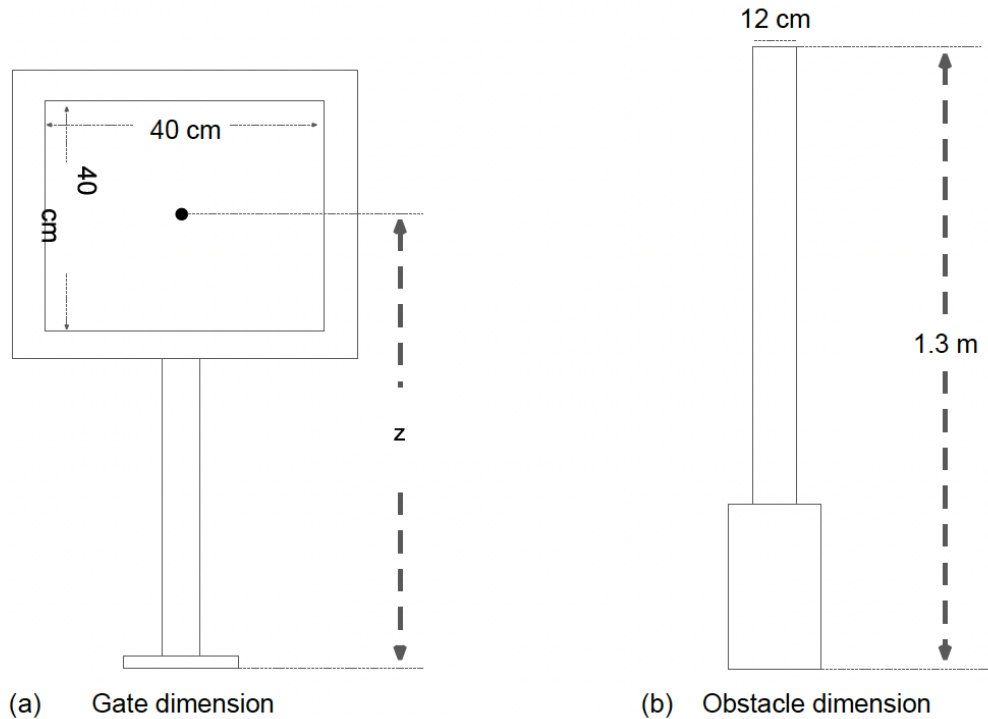


Figure 3: Gate and obstacle dimensions

A complete command enumerate class and corresponding references are provided in `project_utils.py`. You are free to choose either high-level command or a combination of high-level and low-level commands to finish the task. Also, you are free to create your own controller to allow a more accurate and efficient trajectory tracking performance.

Note, a `NOTIFYSETPOINTSTOP` command must be called to transfer drone state from low-level command (`cmdFullState`) to high-level commands (`takeoff`, `land`, `goTo`).

3.4 Requirement

You are required to use the command interface to implement the proposed algorithm in `edit_this.py`. You are required to implement the path planning algorithm on your own instead of using existing packages.

Please maintain the file structure and do not modify `final_project.py` and `project_utils.py` to ensure a direct sim-to-real transfer in the second phase.

Each team will be required to use either:

- search-based algorithms, or
- sampling-based algorithms

for trajectory planning.

4 Final Project Grading

The final project will be graded based on **each individual's performance** and is separated into three parts:

- simulation demonstration (15%)
- experimental demonstration (10%)
- project report (20%)

The project demonstration starts at 2:00 PM, Apr 22nd, 2025, at Myhal 5th floor (Room MY580).

Simulation Performance (15%)

- Pass each of the four gates in simulation, a total of six gates passing required (1.5% each, total 9%)
- Reach the target point in simulation without collision (2%)
- Code (4%)

Experimental Performance (10%)

- Pass each of the four gates in the experiment, a total of six gates passing required (0.5% each, total 3%)
- Reach the target point in experiments without collision (1%)
- Land smoothly after completion of the track (1%)
- Complete the flight within 1 minute (1%)
- Rank of RMSE (root mean squared error) in experiments, the group with the minimal total average RMSE over the path wins (4%), where $RMSE$ is calculated as the equation 1, where the sample points are taken based on the control frequency.

$$RMSE = \sqrt{\frac{\sum_{i=1}^N \|p_i - p_{ref,i}\|^2}{N}} \quad (1)$$

PDF Report (20%)

Maximum of 5 pages (not including title page and references), containing:

- Overview of the algorithms and equations used (7%)
- Discussion of the simulation results (5%)
- Discussion of the difficulties faced (5%)
- Improvements or next steps (3%)

Please submit the code and project report in a .zip format to Quercus by April 28, 2025, 11:59 PM EST.



Good Luck & Have Fun!

